

Пояснение к коду: рекурсивная генерация k-сочетаний (SML)

1. Краткое описание задачи

Задача: сгенерировать все k-сочетания (комбинации) из заданного списка элементов (строк)

Входные данные: список элементов (например ["a","b","c","d"]) и целое k — размер сочетания.

Выходные данные: файл combinations.txt, в котором каждая строка — одно сочетание элементов, разделённых пробелом. и записать их в текстовый файл

2. Краткий обзор алгоритма (крупные шаги)

1. Реализовать рекурсивную функцию `combinations`, которая для пары `(list, k)` возвращает список всех сочетаний размера `k` из `list`.
2. В рекурсивном шаге для первого элемента `x` генерировать сочетания, которые включают `x` (вставляя `x` в начало сочетаний размера `k-1` из `xs`), и сочетания, которые не включают `x` (сочетания размера `k` из `xs`), затем объединять результаты.
3. В основной части программы задать `items` и `k`, вызвать `combinations`, затем записать полученные сочетания в файл `combinations.txt`, преобразовав каждое сочетание в строку с пробелами.

3. Объяснение каждого блока и функции (сверху вниз)

3.1 Функция `combinations`

Код:

```
fun combinations ([] : string list, 0) = [[]]

| combinations (_, 0) = [[]]

| combinations ([], _) = []

| combinations (x::xs, k) =

  let

    val with_x = map (fn comb => x::comb) (combinations(xs, k-1))

    val without_x = combinations(xs, k)

  in
```

`with_x @ without_x`

end; Объяснение:

Пояснение по случаям сопоставления с образцом (pattern matching):

- `fun combinations ([] : string list, 0) = [[]]`
Базовый случай: если список пуст и $k = 0$, результат — список, содержащий одно пустое сочетание `[]`. Это означает, что ровно одно сочетание размера 0 существует — пустая комбинация.
- `| combinations (_, 0) = [[]]`
Базовый случай: если $k = 0$ (независимо от содержимого входного списка), единственное сочетание — пустой список; возвращаем `[]`.
- `| combinations ([], _) = []`
Базовый случай: если входной список пуст, но $k > 0$, невозможно выбрать k элементов — результат пустой список сочетаний `[]`.
- `| combinations (x::xs, k) = ...`
Рекурсивный случай: имеем первый элемент x и остаток списка xs .
 - `val with_x = map (fn comb => x::comb) (combinations(xs, k-1))`
Генерируем все сочетания размера $k-1$ из xs , затем для каждой такой комбинации добавляем x в начало — это все сочетания, которые включают x .
 - `val without_x = combinations(xs, k)`
Генерируем все сочетания размера k из xs — это все сочетания, которые **не** содержат x .
 - `with_x @ without_x`
Объединяем (конкатенируем) списки сочетаний: сначала те, которые содержат x , затем те, которые не содержат. Результат — все сочетания размера k из исходного списка.

3.2 Главный блок

Код:

```
val items = ["a", "b", "c", "d"]
```

```
val k = 2
```

```
val combs = combinations(items, k)
```

Объяснение:

- `val items = ["a", "b", "c", "d"]` — задаётся исходный список элементов.
- `val k = 2` — задаётся размер искомых сочетаний (в данном примере — пары).

- `val combs = combinations(items, k)` — вызов функции `combinations`, результатом является список всех сочетаний размера `k` (тип `String list`): например, `[["a", "b"], ["a", "c"], ...]`.

Код:

```
val out = TextIO.openOut "combinations.txt"

val _ = app (fn comb =>
    TextIO.output(out, String.concatWith " " comb ^ "\n")
) combs

val _ = TextIO.closeOut out

val _ = print ("Ready! " ^ Int.toString(length combs) ^ " combinations →
combinations.txt\n")
```

Пояснение по шагам:

- `val out = TextIO.openOut "combinations.txt"`
Открывает (создаёт/перезаписывает) файл `combinations.txt` для записи и возвращает выходной поток `out`.
- `val _ = app (fn comb => TextIO.output(out, String.concatWith " " comb ^ "\n")) combs`
Итерация по списку `combs`: для каждой комбинации `comb` выполняется анонимная функция:
 - `String.concatWith " " comb` — объединяет элементы списка `comb` в одну строку, разделённую пробелами (например `"a b"`).
 - `^ "\n"` — добавляет перевод строки.
 - `TextIO.output(out, ...)` — записывает сформированную строку в файл `out`.`app` (или `List.app`) пробегает по списку и применяет функцию к каждому элементу; возвращаемое значение `_` игнорируется.
- `val _ = TextIO.closeOut out`
Закрывает файловый поток `out`, гарантируя, что данные записаны и ресурсы освобождены.
- `val _ = print ("Ready! " ^ Int.toString(length combs) ^ " combinations → combinations.txt\n")`
Печатает в консоль сообщение о завершении и количестве сгенерированных сочетаний; `length combs` возвращает число сочетаний.